

CSRF Where Token Is Not Tied to User Session

Lab

PortSwigger Web Security Academy

Vulnerability

Cross-Site Request Forgery (CSRF)

Severity

Medium

Description

The application uses a CSRF token to protect a state-changing request, but the token is **not tied to the user's session**.

An attacker can obtain a valid CSRF token from their own session and use that token in a malicious request targeting another authenticated user.

Because the server only checks whether the token is valid, rather than whether it belongs to the victim's session, the CSRF protection can be bypassed.

Exploitation

First, a legitimate state-changing request was captured:

POST /my-account/change-email HTTP/1.1

Cookie: session=ATTACKER_SESSION

email=attacker@example.com&csrf=VALID_TOKEN

The valid CSRF token was obtained from the attacker's own session.

The request was then modified to target the victim:

POST /my-account/change-email HTTP/1.1

Cookie: session=VICTIM_SESSION

email=attacker@example.com&csrf=ATTACKER_TOKEN

The server accepted the request even though the CSRF token belonged to a different session.

Root Cause

The application validates the CSRF token globally instead of associating it with the authenticated user's session.

Insecure logic:

Is token valid?

Yes → Process request

Secure logic:

Is token valid?

↓

Does token belong to current session?

↓

Yes → Process request

No → Reject

Impact

An attacker may be able to perform unauthorized actions on behalf of another authenticated user, including:

- Changing account information
- Modifying account settings
- Performing other sensitive state-changing actions

Remediation

The application should:

- Generate CSRF tokens securely.
- Bind each token to the user's authenticated session.
- Reject tokens belonging to another session.
- Require a valid token for every protected state-changing request.
- Use **SameSite** cookies as an additional defense.

OWASP / CWE

- **OWASP:** A01 — Broken Access Control
- **CWE-352:** Cross-Site Request Forgery

Conclusion

The vulnerability exists because the CSRF token is not associated with the user's session. An attacker can reuse their own valid token to forge requests against another authenticated user.

CSRF tokens should be **unique and properly bound to the user's session**.

Lab Status: Solved